

가상환경

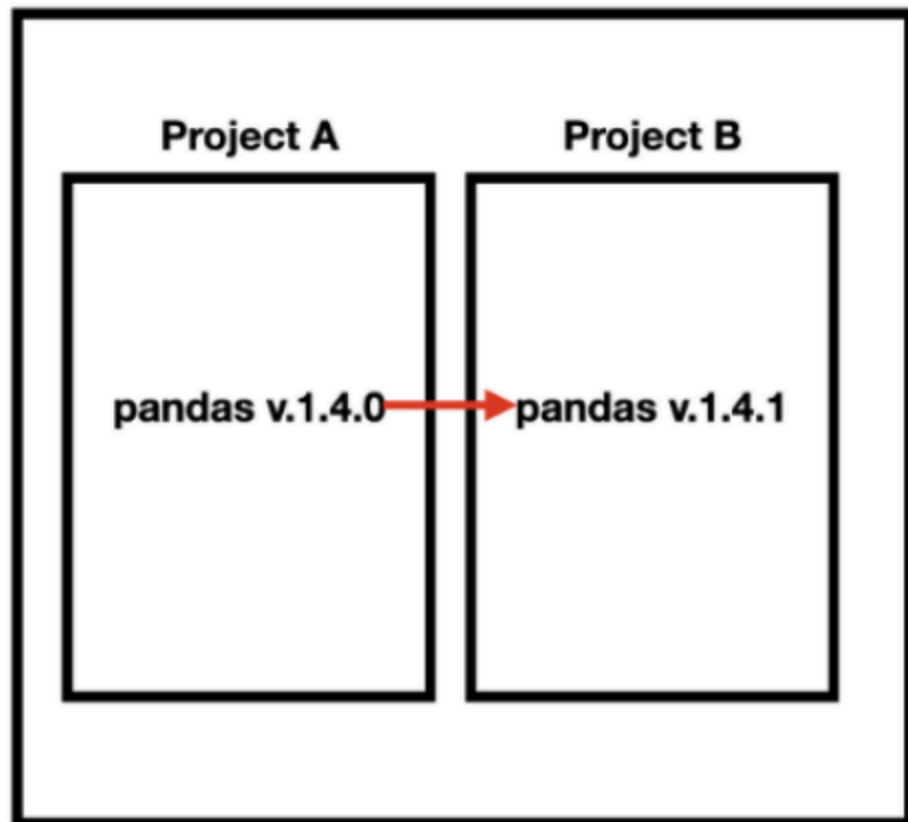
파이썬 가상환경의 주요 목적은 파이썬 프로젝트를 위한 격리된 환경을 만드는 것이다.
(가상환경의 가장 큰 장점은 환경들의 개수에 제한이 없다는 것이다.)

가상환경의 필요성

- 파이썬은 같은 인터프리터 버전끼리 같은 패키지를 공유합니다.
- 예를 들어, 가상 환경을 사용하지 않는 상황에서 A 프로젝트, B 프로젝트에서 Python 3.7.4를 사용하고 가정하겠습니다.
- 그럼 A 프로젝트에서 설치한 패키지를 B 프로젝트에서 따로 설치할 필요 없이 사용할 수 있습니다.
- 프로젝트마다 패키지를 매번 설치하지 않아도 되니 파이썬을 입문하신 분들에게는 매우 편리하게 느껴지실 겁니다.
- 하지만, 프로젝트마다 다른 버전의 패키지를 사용해야 한다면 치명적인 문제를 야기합니다.
- 이를 해결하는 방법이 각 프로젝트별 가상환경을 구축하여 관리하는 것입니다.

Python 3.7.4

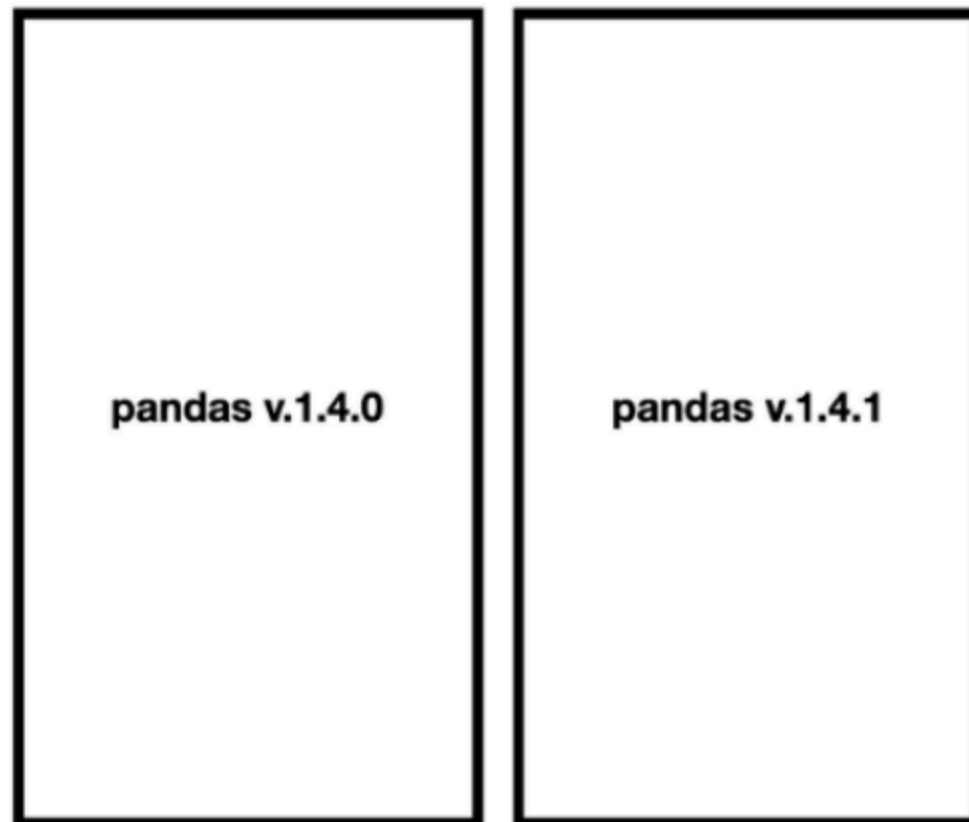
~~pandas v.1.4.0~~ → pandas v.1.4.1



가상환경 사용 **X**

Project A
Python 3.7.4
pandas v.1.4.0

Project B
Python 3.7.4
pandas v.1.4.1



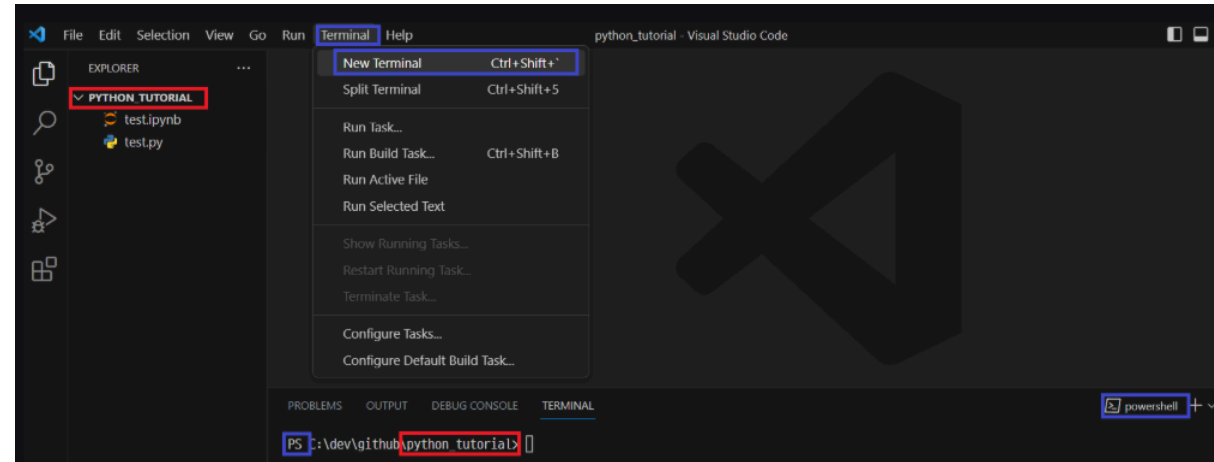
가상환경 사용 **O**

<https://heytech.tistory.com/>

가상환경 생성방법들

1. venv

1. 터미널에서 해당 프로젝트 폴더로 이동
 - 파란색 박스: 터미널 생성하는 방법
 - 빨간색 박스: 프로젝트 폴더 이동 결과 확인 방법

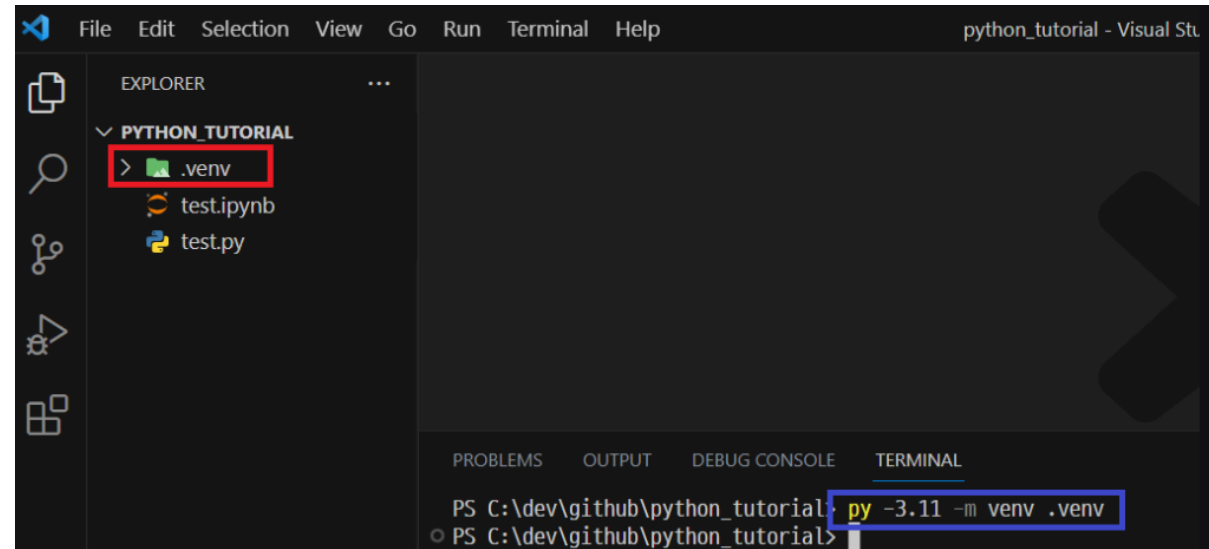


2. 가상환경 생성

- 파란색 박스: 가상환경 생성 명령어

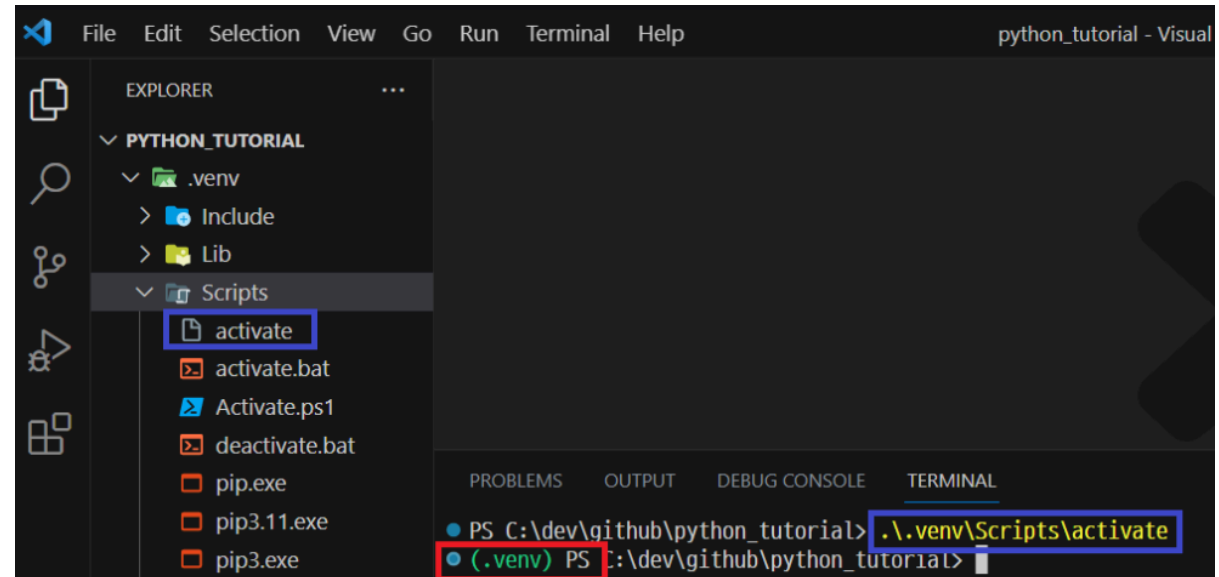
py -파이썬버전 -m venv 가상
환경이름(일반적으로 .venv 사
용)

- 빨간색 박스: 가상환경 폴더



3. 가상환경 접속

- 파란색 박스: 가상환경 접속 명령어
- 빨간색 박스: 접속한 가상환경 확인



2. uv — 현대적인 Python 패키지 매니저

uv는 Rust로 만들어진 초고속 Python 패키지 매니저다.

기존에 사용하던 `pip` + `venv` 조합을 하나의 도구로 대체한다.

기존 방식:	<code>pip + venv + pip-tools</code>	(도구 3개)
uv 방식:	<code>uv</code>	(도구 1개)

얼마나 빠른가?

패키지 설치 속도 비교 (fastapi 기준)

pip	:		20초
uv	:		0.5초 (약 40배 빠름)

uv는 Rust의 병렬 처리와 캐싱 덕분에 pip보다 10~100배 빠르다.

uv 설치

```
# Windows (PowerShell)  
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

```
# Mac / Linux  
curl -LsSf https://astral.sh/uv/install.sh | sh
```

```
# 설치 확인  
uv --version  
# uv 0.x.x (...)
```

가상환경 생성 방식 비교

Python 프로젝트에서 가상환경을 사용하는 이유는 **프로젝트마다 패키지 버전을 독립적으로 관리**하기 위해서다.

uv로 가상환경을 만드는 방법은 두 가지가 있다.

방식 A — 전통적인 pip 스타일 (requirements.txt)

pip에 익숙한 사람이라면 거의 동일한 방식으로 사용할 수 있다.

```
# 1. 가상환경 생성
```

```
uv venv .venv
```

```
# 2. 가상환경 활성화
```

```
## Windows
```

```
.venv\Scripts\activate
```

```
## Mac / Linux
```

```
source .venv/bin/activate
```

```
# 3. 패키지 설치
```

```
uv pip install fastapi uvicorn[standard]
```

```
# 4. 설치된 패키지 목록을 파일로 저장
```

```
uv pip freeze > requirements.txt
```

생성되는 파일:

```
my_project/  
├── .venv/           ← 가상환경 (Git에 올리지 않음)  
├── requirements.txt ← 패키지 목록  
└── main.py
```

다른 PC에서 환경 복원:

```
uv venv .venv  
.venv\Scripts\activate      # Windows  
uv pip install -r requirements.txt
```

방식 B — uv 권장 방식 (pyproject.toml + uv.lock)

uv가 권장하는 현대적인 방식이다. `uv init` 으로 프로젝트를 시작하면 된다.

```
# 1. 프로젝트 초기화
uv init my_project
cd my_project
```

```
# 2. 패키지 추가 (설치 + uv.lock 자동 갱신이 동시에 이루어짐)
uv add fastapi uvicorn[standard]
```

생성되는 파일:

```
my_project/
├── .venv/
├── pyproject.toml
├── uv.lock
└── main.py
```

- ← 가상환경 (**Git**에 올리지 않음)
- ← 프로젝트 설정 & 패키지 목록
- ← 모든 의존성 버전 잠금 파일

다른 PC에서 환경 복원:

```
git clone <저장소 주소>
cd my_project
uv sync          # 끝! .venv 자동 생성 + 동일한 버전 설치
```

방식 A와 B의 복원 명령어 비교:

	방식 A (requirements.txt)	방식 B (uv.lock)
복원 명령어	<code>uv venv .venv + activate + uv pip install -r requirements.txt</code>	<code>uv sync</code>
버전 재현성	직접 명시한 패키지만 고정	모든 의존성 완전 고정
팀 협업	환경 차이 발생 가능	100% 동일 환경 보장

(옵션) Docker에서 uv 사용하기

```
FROM python:3.12-slim
```

```
# uv 설치
```

```
COPY --from=ghcr.io/astral-sh/uv:latest /uv /usr/local/bin/uv
```

```
WORKDIR /app
```

```
# 의존성 파일만 먼저 복사 (소스코드보다 변경이 적음)
```

```
COPY pyproject.toml uv.lock ./
```

```
# 패키지 설치
```

```
RUN uv sync --frozen --no-dev --no-install-project
```

```
# 소스코드 복사
```

```
COPY . .
```

```
CMD ["uv", "run", "uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

주요 옵션 설명

옵션	설명
<code>--frozen</code>	<code>uv.lock</code> 이 변경되었으면 빌드 실패 → 의도치 않은 버전 변경 방지
<code>--no-dev</code>	개발용 패키지 제외 → Docker 이미지 용량 감소
<code>--no-install-project</code>	소스코드 제외, 의존성만 먼저 설치 → 캐시 효율 향상

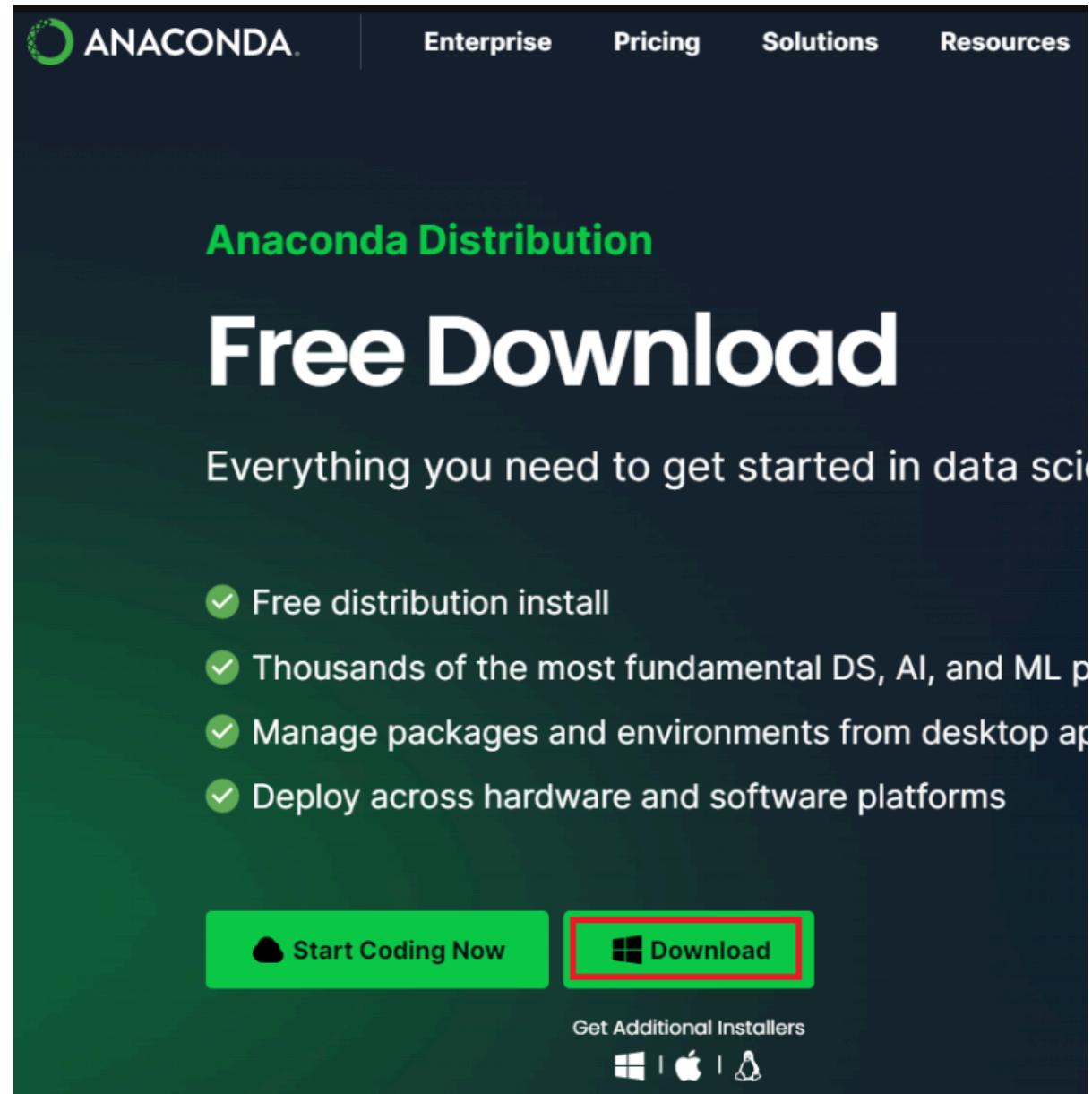
자주 쓰는 uv 명령어 정리

명령어	설명
<code>uv init <프로젝트명></code>	새 프로젝트 생성
<code>uv add <패키지></code>	패키지 추가 & 설치
<code>uv remove <패키지></code>	패키지 제거
<code>uv sync</code>	uv.lock 기준으로 환경 동기화
<code>uv run <명령어></code>	가상환경 활성화 없이 명령어 실행
<code>uv pip install -r requirements.txt</code>	기존 방식으로 설치 (호환용)
<code>uv pip freeze</code>	현재 설치된 패키지 목록 출력

2. Anaconda

- Anaconda는 데이터 과학(Data Science), 머신러닝(ML), 인공지능(AI) 개발을 위해 필요한 파이썬 환경을 한 번에 제공하는 배포판(Distribution)입니다.
- "파이썬 + 패키지 관리자(conda) + 데이터 분석 라이브러리 + 개발 도구"를 묶어놓은 올인원 패키지

1. 아나콘다 다운로드



The image shows the top section of the Anaconda website. At the top is a dark blue header with the Anaconda logo and navigation links: Enterprise, Pricing, Solutions, and Resources. Below the header, the text 'Anaconda Distribution' is in green, followed by 'Free Download' in large white letters. A subtitle reads 'Everything you need to get started in data science'. A list of four features with green checkmarks is shown: 'Free distribution install', 'Thousands of the most fundamental DS, AI, and ML packages', 'Manage packages and environments from desktop applications', and 'Deploy across hardware and software platforms'. At the bottom are two green buttons: 'Start Coding Now' and 'Download'. The 'Download' button is highlighted with a red rectangle. Below the buttons, it says 'Get Additional Installers' with icons for Windows, macOS, and Linux.

ANACONDA

Enterprise Pricing Solutions Resources

Anaconda Distribution

Free Download

Everything you need to get started in data science

- ✓ Free distribution install
- ✓ Thousands of the most fundamental DS, AI, and ML packages
- ✓ Manage packages and environments from desktop applications
- ✓ Deploy across hardware and software platforms

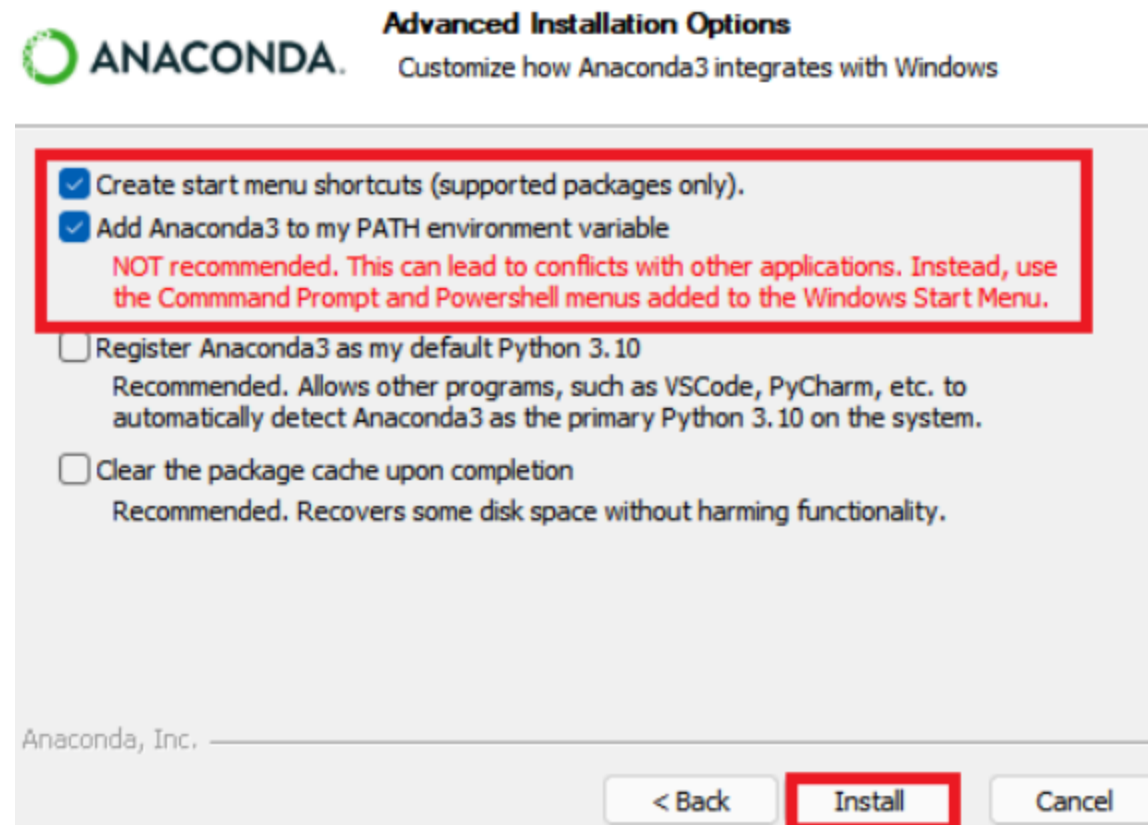
 Start Coding Now  Download

Get Additional Installers

 |  | 

2. 아나콘다 설치

PATH를 추가하는 팝업창이 나오면, 아래와 같이 Anaconda를 시스템 환경변수에 추가해 준다.



가상환경 생성

```
conda create -n 가상환경 이름 python=파이썬 버전
```

```
$ conda create -n velog python=3.8
```

- 나는 python version이 3.8인 velog라는 이름을 가진 가상환경을 생성해 보았다.

가상환경 활성화 & 비활성화

```
conda activate 가상환경 이름
```

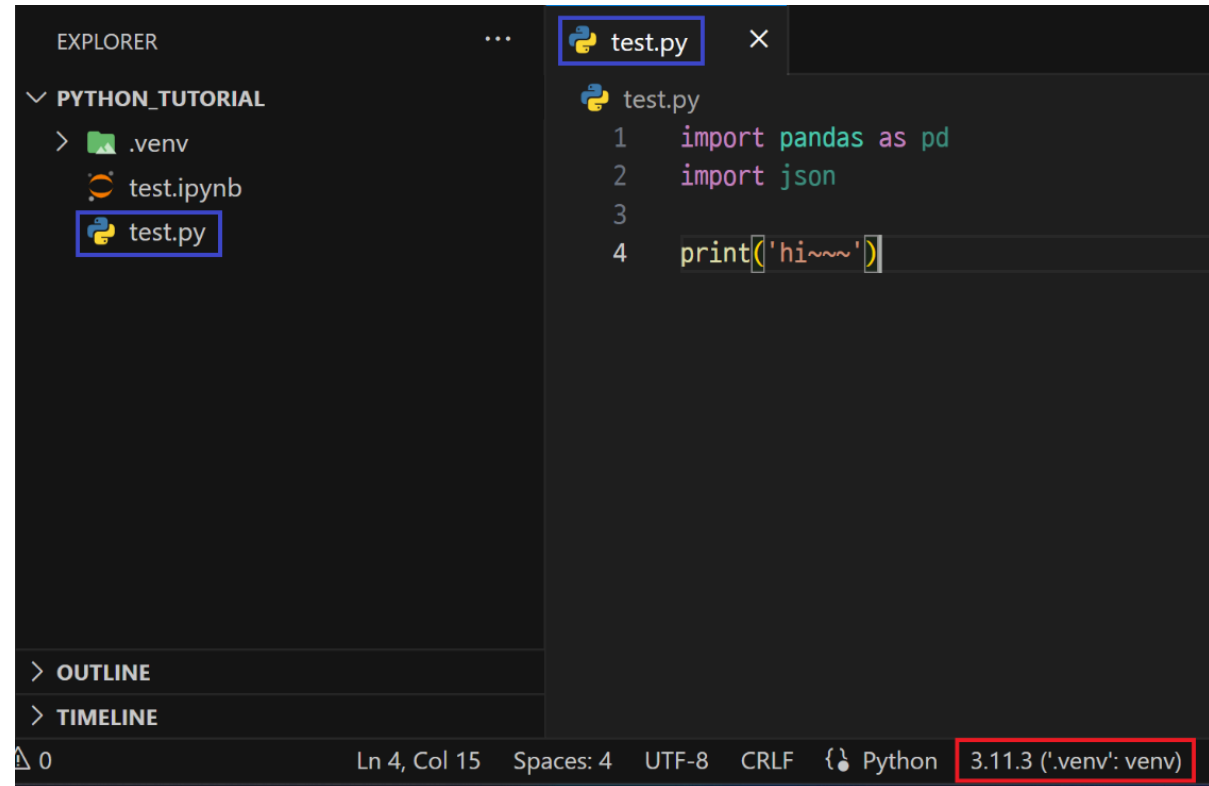
```
$ conda activate velog
```

- 가상환경을 이용하기 위해서 가상환경을 활성화 시켜주었다.

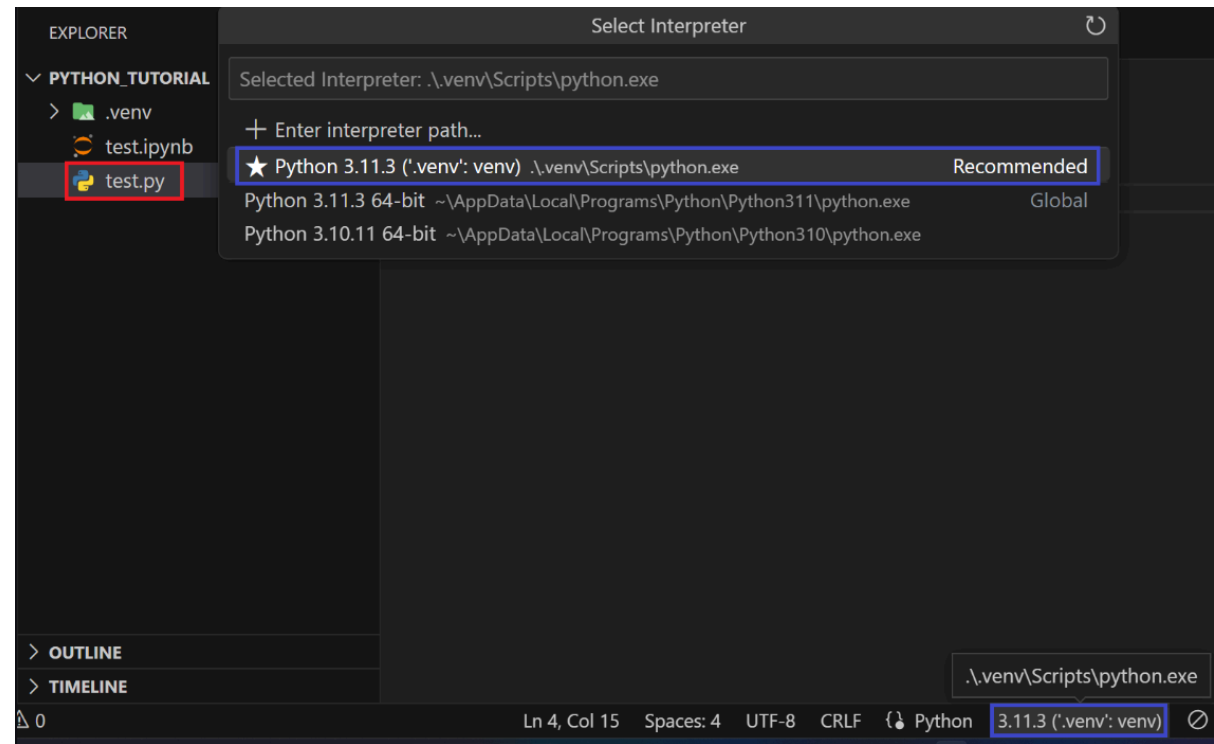
VSCode에서 가상환경 적용하기

python 파일

- 빨간색 박스: 가상환경을 확인할 수 있습니다.

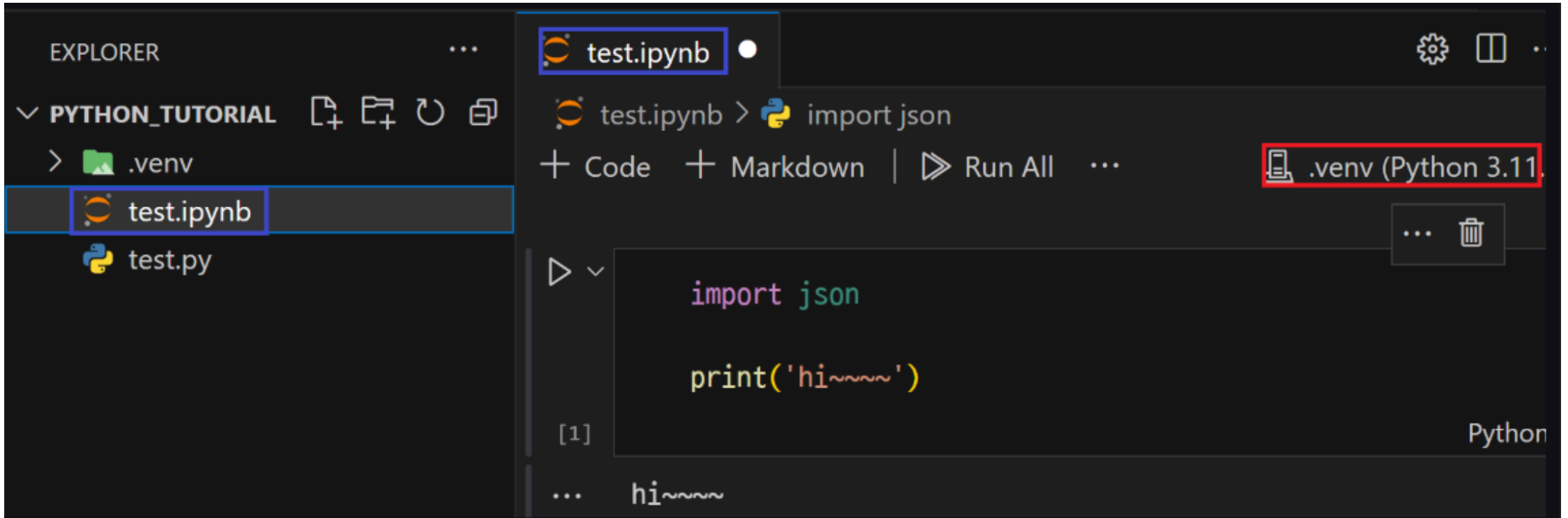


- 파란색 박스: 가상환경을 변경할 수 있습니다.



jupyter 파일

- 빨간색 박스: 가상환경을 확인할 수 있습니다.



- 파란색 박스: 가상환경을 변경할 수 있습니다.

